

一文读懂上下文工程(Context Engineering)

mp.weixin.qq.com/s/8PCY7lvZhCDJtXiDpx8bOQ

J杰克

Original



摘要：本文以Claude最新的关于有效上下文管理的论文为基础，结合几篇与上下文工程(Context Engineering)相关的文章，整理了所有与上下文工程相关的概念，主要以说明下面三个问题：

- 什么是上下文(Context)?
- 为什么要进行上下文工程(Context Engineering)?
- 如何进行上下文工程(Context Engineering)?

让我们开始吧！

最近在深度使用Claude Code作为主力编程工具，工作过程中，与自己原来使用的github copilot、Cursor以及Trae等AI Coding工具对比，效果非常好。看过一些资料讲Claude Code在上下文管理上的特点，但是都没有讲得太透。

节前，Anthropic发布了一篇研究博文 Effective context engineering for AI agents^[1]，从Claude Code开发者的角度深入讲解了他们对上下文工程的理解，很有启发。

结合自己的实际工作，再加上最近看到的一些资料，在本文中以Anthropic的这篇文章为纲，写写自己的理解。

什么是上下文 (Context) ?

大模型(LLM)是一个概率模型，它只能根据你的输入，通过预测下一个token的概率分布，来生成文本。

在深入分析前，有一点需要明白，所有的语言模型都是无状态的(Stateless)。这意味着，模型在生成文本时，只依赖于当前输入的token，而不依赖于之前的token。

但是就像我们人类平时沟通一样，如果没有足够的信息，我们也很难准确地把握对方的意图，这时我们一般会多问几个问题，了解一下前因后果，帮助自己做些判断。

大语言模型也是这样的，它如果你问的问题了解的更多，那么它的回答也更加准确、更有针对性，帮助可能也更大。

而我们添加给模型的额外信息，就叫做上下文 (Context)。大概的形式有以下几种。

用户提示(User Prompt)

我们平时在电脑上或手机上使用豆包、混元、Deekseek时，一般的用法是写一个问题或指令，模型会给予一个应答。



在这个例子中，我先告诉它我的名字，然后再问它"我是谁"，它会按照我们对话的历史进行正确的回复。

而如果我们直接上来问"我是谁"，那么它就没有办法回答了。



我们的提出问题就叫做用户提示(User Prompt)。

而在多轮对话中，User Prompt与模型的回答，共同组成对话历史，它是上下文构成的基础。

除了对话的历史(History)，为了让大模型的输出更加有效和有帮助，我们也可以对User Prompt进行优化，从而让模型更好地理解我们的意图和需求。

而优化的方法，其实就是在User Prompt中添加一些额外信息，让模型了解更多的语境（其实就是**上下文Context**）。

比如，界定受众的人群，让它按我们要求的方式回复：

好的，这是一个关于太阳系的简单介绍。

太阳系是我们所在的行星系，它以**太阳**为中心，所有天体都在太阳的巨大引力作用下运行。

我们可以将太阳系的主要成员分为几个类别：

1. 太阳 - 系统的中心

太阳是一颗巨大的、发光发热的**恒星**，它占据了太阳系总质量的**99.86%**。它的强大引力束缚着所有其他天体，并且是太阳系中光和能量的主要来源。

2. 八大行星

行星是围绕太阳运行、自身不发光、且能清除其轨道附近其他物体的天体。它们通常分为两类：

- **类地行星（内行星）**：靠近太阳，由岩石和金属构成，体积较小，表面坚实。
 1. **水星**：距离太阳最近，表面布满陨石坑，昼夜温差极大。
 2. **金星**：大小与地球相似，但有着浓密的酸性大气层，是太阳系最热的行星。
 3. **地球**：目前已知唯一有生命存在的星球，表面有液态水和适宜的大气。
 4. **火星**：红色的行星，表面有沙漠、峡谷和极冠，是人类探索的重点。
- **类木行星（外行星）**：离太阳较远，体积巨大，主要由气体（氢、氦）和冰组成，没有坚实的表面。
 5. **木星**：太阳系中最大的行星，是一个气体巨行星，有著名的大红斑（一个巨大的风暴气旋）。
 6. **土星**：以其美丽而明亮的**行星环**闻名，这些环主要由冰和岩石碎片组成。
 7. **天王星**：一个“冰巨行星”，其自转轴几乎与公转轨道平行，像是“躺着”公转。
 8. **海王星**：太阳系最边缘的行星，一个蓝色的冰巨行星，风速极快。

3. 其他重要成员

- **矮行星**：与行星类似，但未能清除其轨道附近的其他天体。最著名的代表是**冥王星**，以及位于小行星带的**谷神星**等。
- **小行星**：主要分布在火星和木星轨道之间的小行星带，是岩石和金属构成的小天体，形状不规则。
- **彗星**：由冰、尘埃和岩石组成的“脏雪球”。当它们靠近太阳时，冰物质升华，形成长长的**彗尾**，非常壮观。
- **卫星**：围绕行星运行的天体。太阳系中有数百颗卫星，比如地球的**月球**、木星的**木卫二**、土星的**土卫六**等，它们本身就是一个丰富多彩的世界。

一个重要的边界：柯伊伯带

在海王星轨道之外，有一个由冰质小天体组成的广阔圆盘状区域，称为**柯伊伯带**。冥王星就位于这里，可以看作是柯伊伯带中最著名的天体之一。

总结一下：

太阳系是一个以太阳为中心，由八大行星、数以亿计的小行星、彗星、矮行星和卫星等天体组成的庞大“家族”。它广阔而复杂，虽然我们已经了解了很多，但仍有无数奥秘等待探索。

希望这个简单的介绍能帮助你建立一个对太阳系的基本概念！

向一个五岁的孩子简单介绍一下太阳系



当然！宝贝，我们来想象一下太阳系就像一个超大的游乐园。

1. 太阳是游乐园中心的大灯泡💡

它是一个超级大的、会自己发光发热的大火球。它待在正中间，给我们光和热，让地球变得暖融融的。

2. 行星是围着灯泡转的小汽车🚗

有八辆不同颜色和样子的小汽车，沿着自己的轨道，一圈一圈地围着大灯泡开。

- **跑得最快的小汽车——水星**：离灯泡最近，它跑得最快！

- 蒙着被子的小汽车——金星：它被厚厚的云层被子裹着，是世界上最热的星球。
- 我们的家——地球🌍：我们住在这辆蓝色和绿色的小汽车上！它有水 and 空气，还有你和所有的小动物。
- 红色的小汽车——火星：它是红色的，像草莓一样。科学家叔叔想看看那里有没有外星小怪兽。
- 最大最胖的国王——木星：它是所有小汽车里最大最胖的，身上有个超级大的“大眼睛”（其实是个大风暴）！
- 戴着草帽的明星——土星🪐：它最漂亮，有一个闪闪发亮的呼啦圈（光环）！
- 躺着打滚的淘气包——天王星：它很淘气，是躺着打滚转圈的。
- 蓝色的风娃娃——海王星：它是蓝色的，离灯泡最远，那里的风刮得呼呼的，超级大！

3. 还有其他好玩的东西

- 小石头（小行星）：在火星和木星之间，有很多很多小石头，它们也在一起玩。
- 脏雪球（彗星）：有时候会有一个拖着长长亮尾巴的“脏雪球”飞过来，像冰淇淋一样，可好看啦！

简单来说：

太阳系就是一个大火球（太阳）和八个围着它转的星球小汽车，我们呢，就住在叫做地球的这辆小汽车上！

看，这就是我们的太阳系大家庭，是不是很神奇呀？🌟

📄 🔄 📌 🗨️ 📶

👤 公众号 · LiveThinking

在没有界定"5岁孩子"这个语境时，它的输出只是非常普通的科普文，而给5岁孩子讲解时表达要更加活泼有趣。

系统提示(System Prompt)

如上所述，通过对话历史可以让大语言模型更加了解我们，而如果我们想让模型只按某种角色与我们交流，那么这里我们可以使用系统提示(System Prompt)。

现在大多数工具（豆包、混元、Deekseek等）不支持用户自定义的系统提示，只能使用默认的系统提示。而Gemini、OpenAI等也只是在开发者环境可以使用系统提示(System Prompt)。

但是，我们可以在对话开始时，先做一个基础设定，那么后续的很多回复都可以基于这个设定，它大致能起到系统提示的作用。

如，在Deepseek中，对电影分类的表述进行情绪的判断：

根据我的输入来判断我语气的情绪，按如下的要求回复：

例子1

- 分类: '这是我看过的最好的电影!'
- 回复: 积极

例子2

- 分类: '我不喜欢这部电影，简单是浪费时间!'
- 回复: 消极

例子3

- 分类: '情节不出所料，但演技还行。'
- 回复: 中性

准备好后，提醒用户输入新的分类。

好的，我已经理解了任务要求。现在请提供需要分类的文本，我会根据情绪判断返回“积极”、“消极”或“中性”的评估结果。

分类: '这部电影还行，不好也不坏!'

中性

公众号 · LiveThinking

在这个例子中，我们在第一条消息中，进行了一些基础的设定（这个是在通用工具中使用系统提示的一种折衷的方法），并使用少样本(Few Shot)方式，提供了一些说明，要求模型只按我们的要求回复（不要输出一大堆说明性的文字），那么它就会按我们的要求进行回复。

其实在Claude Code等coding agent中，后面会有更复杂的系统提示进行驱动。

消息历史(Messages History)

上面我们已经讲过在对话过程中，每次交互的组合形成了这次对话的历史信息。

再次提示：大模型是没有记忆的，它只是接受每次的请求，然后进行应答而已。

那么它是怎么了解到我们聊天的历史信息呢？

其实都是要我们组织好传送给它。

以Deepseek接口(它兼容OpenAI的接口标准，适用于大多数模型)的为例，在每次交互时，在接口中都会有一个messages字段，接口文档的定义如下：

Request

APPLICATION/JSON

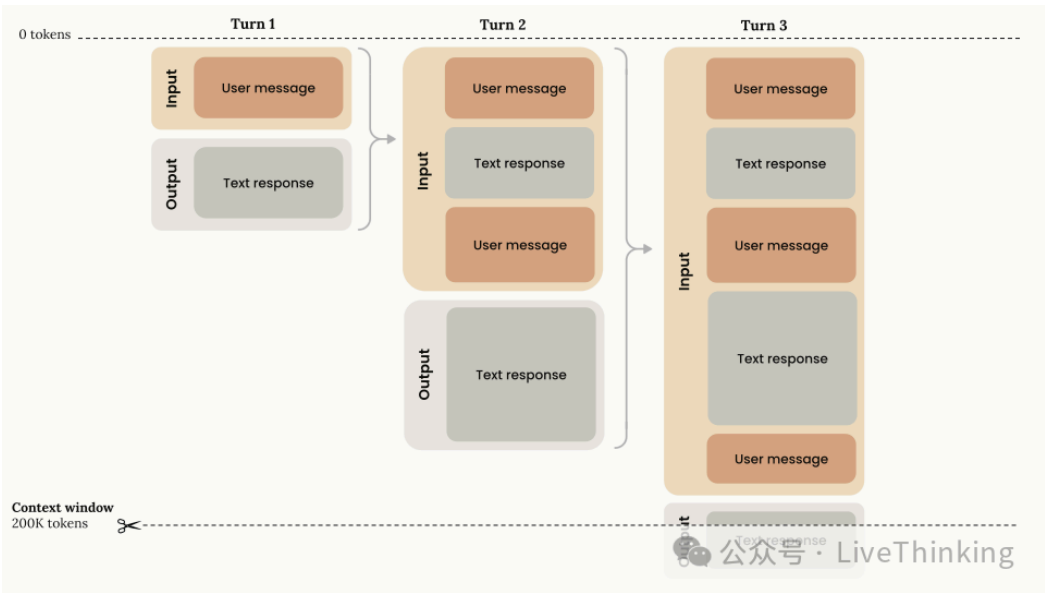
▼ BODY REQUIRED



`messages`是一个数组，它包含以下几类信息：

- 1. 1. System message。就是系统提示，每次调用时都需要上传给大模型。
- 2. 2. User message。就是用户提示。
- 3. 3. Assistant message。这个是模型的回答，每个User message会对应一个Assistant message。
- 4. 4. Tool message。这个是模型调用工具的回答，每个Tool call会对应一个Tool message。在后面会讲解。

一个完整的`messages`数组，就像下面这样：



摘自Claude API文档

这些所有的消息都需要开发人员在调用接口时每次都要重新上传给大模型。

这种会话的历史，一般是按用户登录后的会话进行保存，这样就可以在会话中每次将这些信息传送给大模型。

会话(Session)在技术上一般与用户的登录状态绑定的，在用户退出后，就不会再保存了，这个称为短期记忆(Short-term Memory)。

这些信息也可以保存到后台的数据库中，从而实现长期记忆(Long-term Memory)。即以后可以根据用户的信息读取。

工具信息(Tool Use)

现在的大模型在一般回复时，都是使用预训练中的内部知识集，这些知识一般是有截止时间的，比如，下面的Deepseek：

你的知识是截止到什么时候的？

📄 ✎

我的知识截止日期是 **2024年7月**。

这意味着：

- 我能很好地理解和回答关于**2024年7月及之前**发生的事件、存在的信息和数据。
- 对于**2024年7月之后**发生的事件、发布的新研究、新闻等，我无法提供信息。

举个例子：

- ☒ 我可以告诉你2024年巴黎夏季奥运会的情况（因为它在我知识截止前已规划好）。
- ☐ 但我无法告诉你2024年巴黎奥运会的比赛结果或哪位运动员获得了金牌（因为这些具体事件发生在我的知识截止之后）。

重要提示：

虽然我的核心知识库截止到2024年7月，但当我通过联网搜索功能获取信息时，可以突破这个时间限制，为你提供最新的新闻和数据。不过，你需要手动在Web/App上点击联网搜索按键才能开启这个功能。

所以，在问我问题前，特别是关于近期事件时，你可以先判断一下是否需要我进行**联网搜索**来获取最新信息。😊

📄 🔄 📌 🗨️ ⬆

👤 公众号 · LiveThinking

deepseek最新版本的知识截止日期

最新版本的deepseek的内置的预训练知识是截止2024年7月的。你如果直接问它“现任的美国总统是谁？”，它会回答你是“拜登”（有兴趣可以自己试试）

在这种情况下，模型都是利用内部的信息进行回复的。而通过增加一些工具，就可以让大模型与外界进行沟通，从而获取一些最新的知识信息。

这就要使用到工具调用，即Tool Use。

现在所有的模型都会支持一些标准的工具，如代码执行、在线搜索等。

还有，现在的大模型都是文字模型，它可以做一些数学计算的工作，但是对于复杂的运算，它会出错，如下：



大模型数学计算能力

这里我们就需要给大模型提供工具来帮助大模型做一些工作。

比如，在deepseek中，我们可以打开"联网搜索"，问刚才那个同样的问题，如下：



联网搜索

可以看到deepseek会自动去网络上查询最新的信息，并且准确回复。

这个调用的过程，有些模型会开放给用户(如deepseek)，而有些模型不会开放给用户(如OpenAI等)，但是这些工具调用的过程生成的内容也是上下文的一部分。

现在流行的MCP(Model Context Protocol)本质上就是使用Tool Use的标准协议，使用MCP，各种服务都可以通过工具的形式提供给模型，而模型根据实际的情况进行调用。

外部知识(External Knowledge)

我们可以有两种方式将外部知识提供给模型：

1. 预先准备好外部知识，如先查询数据库，然后将查询到的信息放在用户提示(User Prompt)中，提供给大模型，这是传统RAG(Retrieval-Augmented Generation)的方式。
2. 让大模型在回答问题的过程中，自己去搜索必要的知识，这里使用的是特定的Tool Use工具。然后，这些Tool Use过程的信息就会放在messages作为上下文提供给大模型。这是Agentic的方式。

由于使用的方式都上面已经讲过，这里不再赘述。

推理信息(Reasoning Message)

去年年底，OpenAI O1的推出，将推理能力提升了一个台阶。而今年2月，deepseek将自己实现的推理过程开放给用户，结果整个业界带来了很大的震动。

现在几乎所有的主流模型在回复问题前，都会提供大段大段的推理信息，比如，



已深度思考（用时 15 秒）

已阅读 8 个网页

嗯，用户想了解中美关税战的最新情况，要求简要回复。这是一个关于当前国际经贸热点的时效性问题，我需要从搜索结果中找出最新、最核心的信息来回答。

首先我注意到多个搜索结果 1 3 6 都提到特朗普在2025年10月10日宣布，将从11月1日起对中国商品额外加征100%关税。这是最核心的最新动态，必须放在回答最前面。这些关税将叠加在现有约57%的关税基础上 3 6，使总关税水平可能达到130%或更高 1，甚至有些来源提到150%以上 6。

同时特朗普还宣布将对所有关键软件实施出口管制 3 8，这也是重要信息。关于这次关税升级的直接原因，搜索结果 3 8 显示是对中国稀土及相关技术出口管制措施的回应，这点也需要包含。

市场反应方面，2 8 详细描述了美股因此出现的暴跌情况，特别是科技股和中概股遭受重创，这是关税战影响的直接体现，应该简要提及。

中方立场部分，5 提到中国商务部批评美方“双重标准”，并指出美方在短期内持续推出对华限制措施，破坏谈判氛围。这个表态很重要，能体现双方立场对立。

我还注意到 4 提到一个重要细节，就是特朗普的声明目前还缺乏法律程序支撑，白宫尚未发布行政命令，这暗示政策可能还有变数，值得在回答中说明。

考虑到用户要求“简要回复”，我会把这些关键信息浓缩成几个核心要点，按时间顺序和逻辑关系组织，确保回答简洁但信息完整。

根据2025年10月中旬的最新信息，中美关税战形势因美国总统特朗普的一项新声明而骤然紧张。

最新动态概览

下表整理了最核心的事件与各方反应：

方面	主要内容
美方行动	美国总统特朗普于10月10日宣布，计划从2025年11月1日起，对所有中国进口商品额外加征100%的关税 1 6 8。这将叠加在现有税费上，使美国对华平均关税从约57% 飙升至130% 甚至150% 以上 1 3 6。同时，美方还将对“所有关键软件”实施出口管制 3 8。
直接起因	此举被普遍视为对中国此前发布稀土及稀土技术出口管制决定的回应 3 5 8。
市场反应	该声明引发全球金融市场恐慌，导致美国股市大幅暴跌，科技股与中概股遭受重创 2 8。
中方回应	中国商务部新闻发言人指出，美方表态是典型的“双重标准”，并强调美方在短期内持续推出对华限制措施，严重破坏双方经贸会谈氛围 5。

总结

目前，特朗普的加税声明还处于威胁阶段，最终是否会落地以及具体细节仍存不确定性 3 4。这一事件标志着中美贸易战在经历了一段谈判与缓和期后，再度面临急剧升级的风险。

希望以上信息能帮助你快速了解当前局势。

本回答由 AI 生成，内容仅供参考，请仔细甄别。

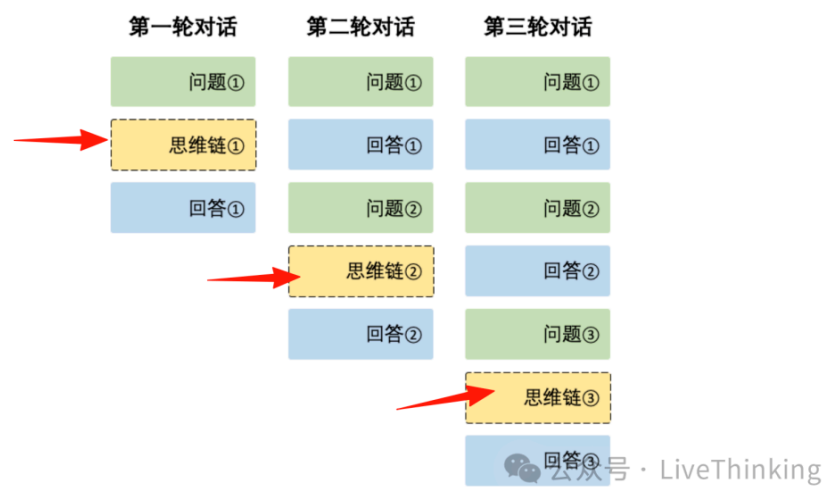
8 个网页

公众号 · LiveThinking

在大模型处理时，一般会要求不要将推理信息放在上下文(Context)中，

上下文拼接

在每一轮对话过程中，模型会输出思维链内容（reasoning_content）和最终回答（content）。在下一轮对话中，之前轮输出的思维链内容不会被拼接到上下文中，如下图所示：

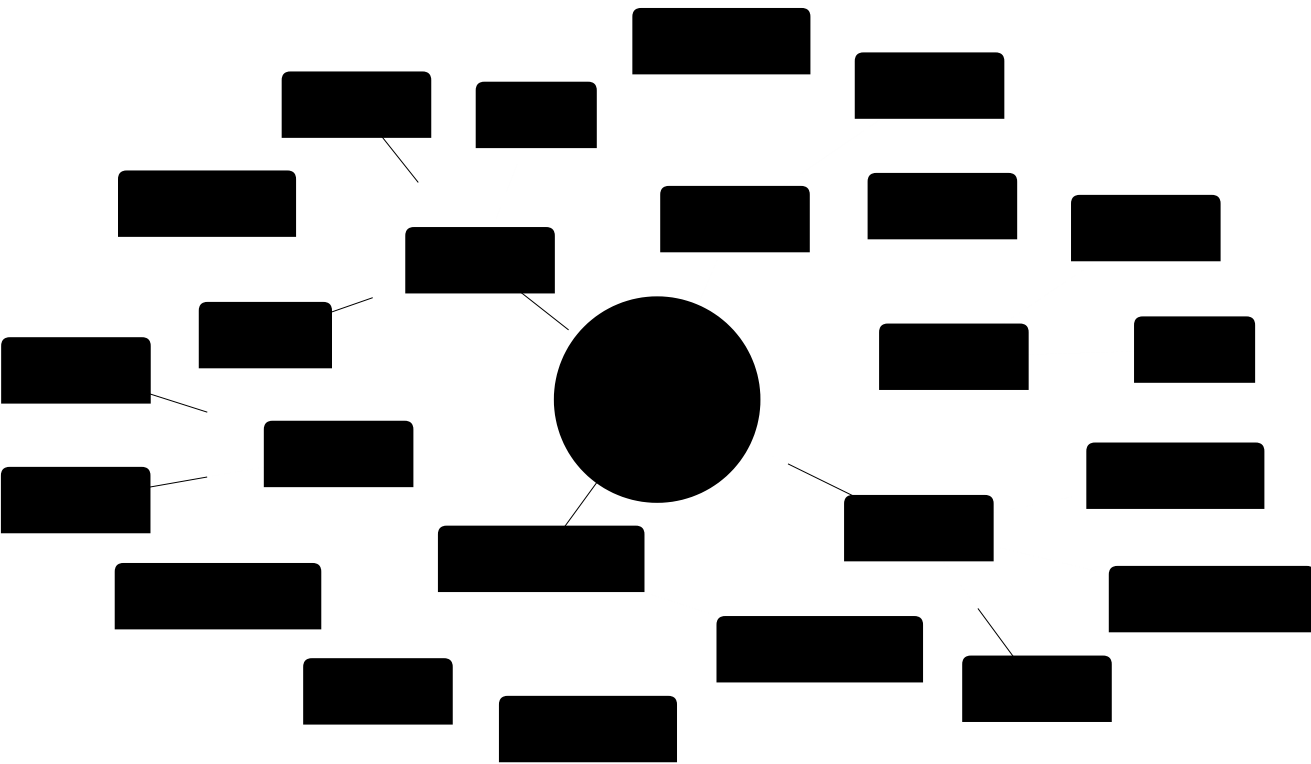


摘自Deepseek API文档

我们一般不会将这个推理过程放在上下文中，但是大家要知道有这样的信息产生。

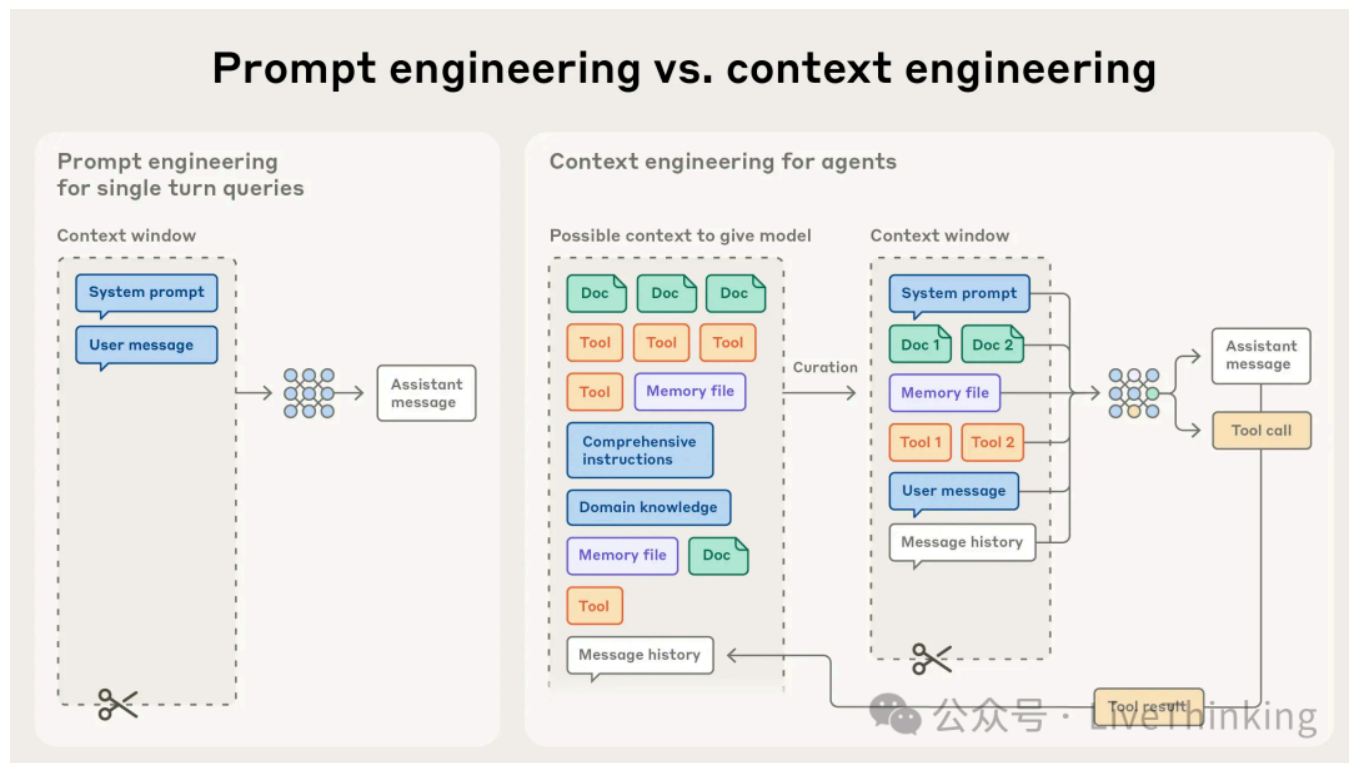
上下文总结

根据上面的分析，我们有这几类信息组成了上下文的内容：



在Claude的博文中，通过上下文工程(Context Engineering)与提示工程(Prompt Engineering)的比较，

Prompt engineering vs. context engineering



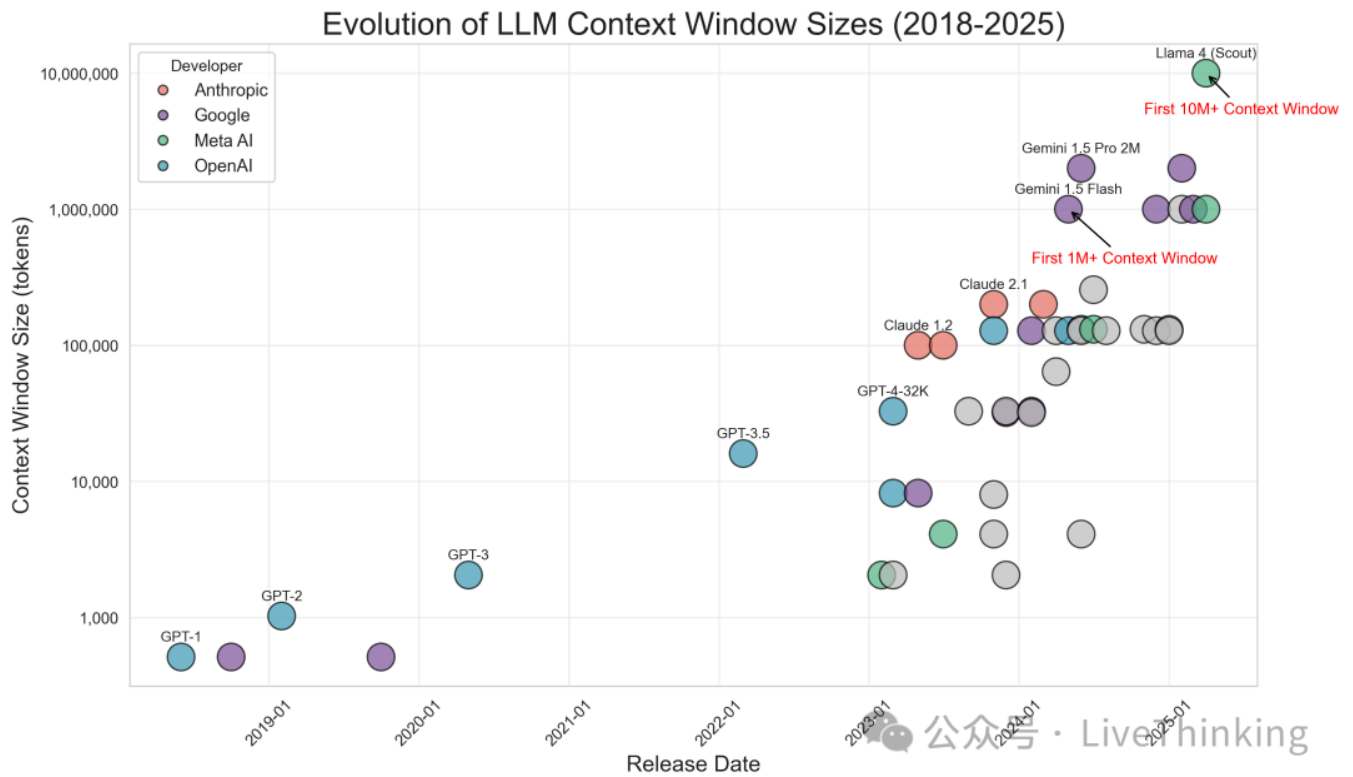
摘自Claude博文

与编写提示这一离散任务相比，上下文工程是迭代的，并且每次我们决定要传递什么给模型时，都会进行策划阶段。

为什么要进行上下文工程（Context Engineering）？

上下文长度(Context Length)是有限的

大模型的上下文已经从最初的几K Tokens增加到现在的上百万 Tokens:



图片来源: <https://www.meibel.ai/post/understanding-the-impact-of-increasing-llm-context-windows>

下表是一些主流模型上下文窗口长度的比较：

模型名称	上下文长度 Context Length
OpenAI GPT-5	400,000
OpenAI GPT-4.1	1,047,576
OpenAI O4 mini	200,000
OpenAI O3	200,000
Google Gemini 2.5 Pro	1,048,576
Google Gemini 2.5 Flash	1,048,576
Claude Sonnet 4.5	200,000
Claude Sonnet 4	200,000
Deepseek	128,000
GLM-4.6	200,000

现在的主流大模型的上下文长度最大已经达到 1M Tokens。

但是，对于复杂的页面来讲，比如今年爆发的 Agent 应用，它往往会有多轮模型自主的工具调用，这意味着，上下文很快就会出现问题的。

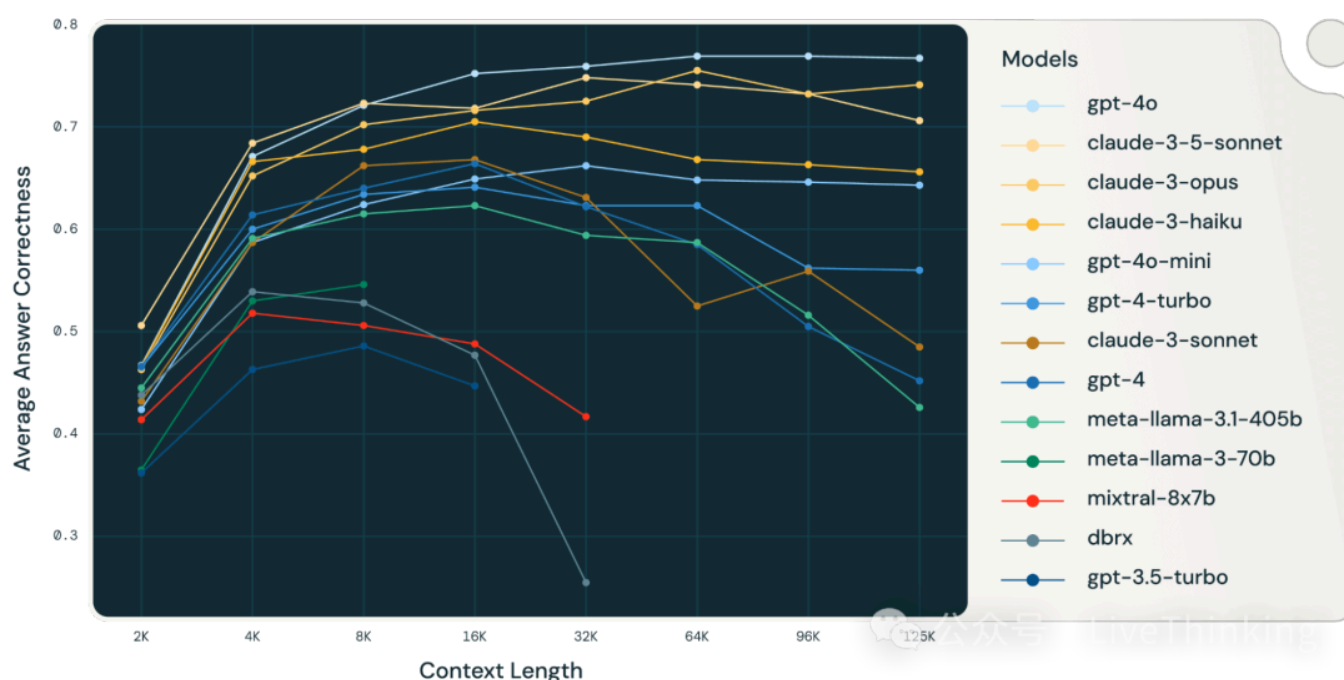
所以，上下文工程(Context Engineering)最核心的目标是在任务执行的过程中避免塞爆上下文空间。

上下文随着长度增加而变得复杂

在Claude的博文一开始就提到了这一点：

尽管 LLM 速度快，并且能够管理越来越大的数据量，但我们观察到，LLM 和人类一样，在某一点上会失去焦点或感到困惑。关于“大海捞针(needle-in-a-haystack)”式基准测试的研究揭示了上下文腐烂(context rot)的概念：随着上下文窗口中 token 数量的增加，模型从该上下文中准确回忆信息的能力会下降。

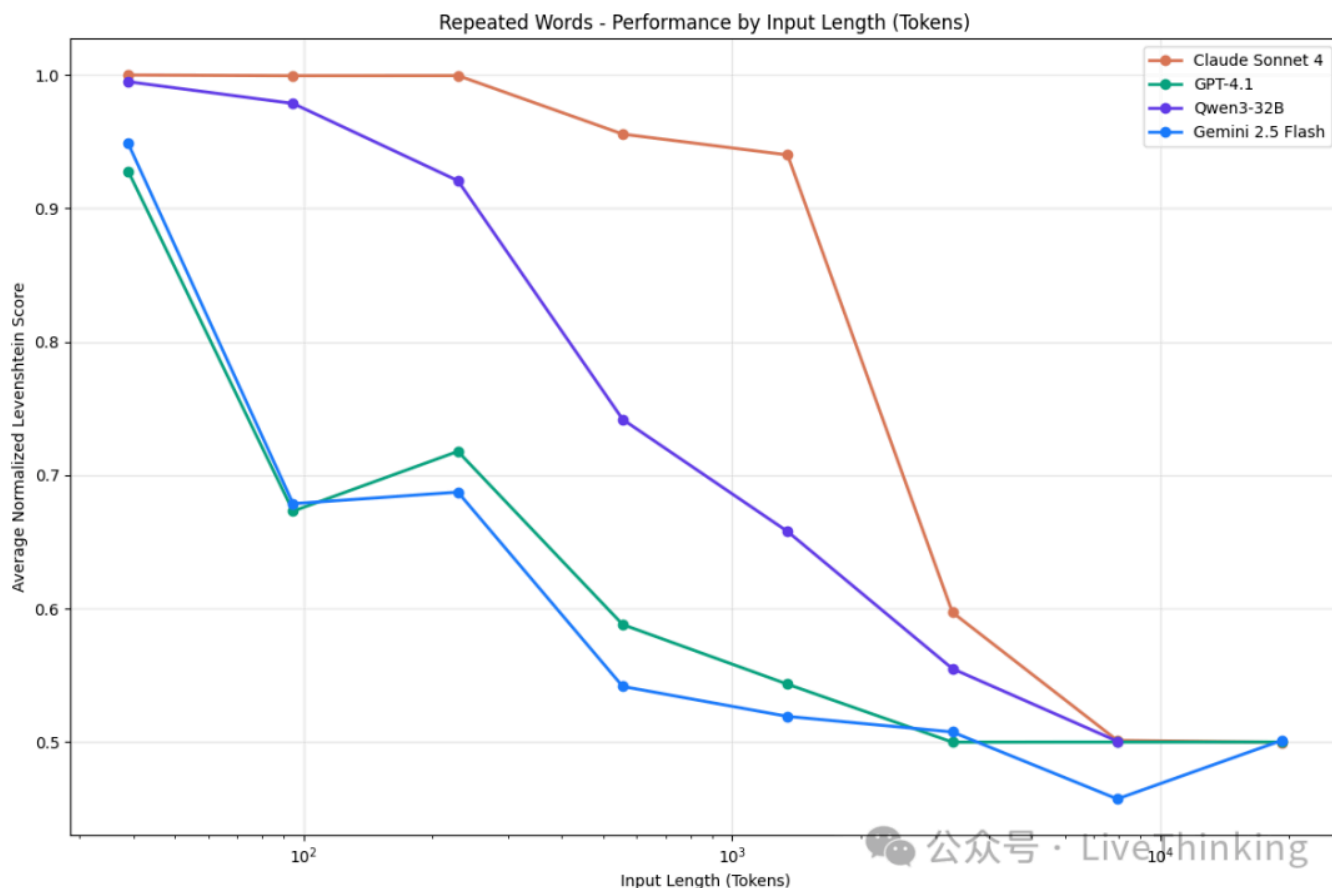
这说明了虽然上下文窗口越大，模型能够处理的信息就越多，但是，上下文必须被视为一种边际收益递减的有限资源。



上下文长度与回答质量的关系，引自：<https://www.databricks.com/blog/long-context-rag-performance-llms>

也就是你虽然可以给模型塞100万 Tokens，但是很多时候，也就意味着它的回答质量的下降。

还有，在这项Chroma关于模型上下文内容研究^[2]中，评估了 18 个 LLM，包括最先进的 GPT-4.1、Claude 4、Gemini 2.5 和 Qwen3 模型。研究的结果显示，模型并不能一致地使用其上下文；相反，随着输入长度的增长，它们的性能变得越来越不可靠。



上下文内容与回答质量的关系，引自：<https://research.trychroma.com/context-rot>

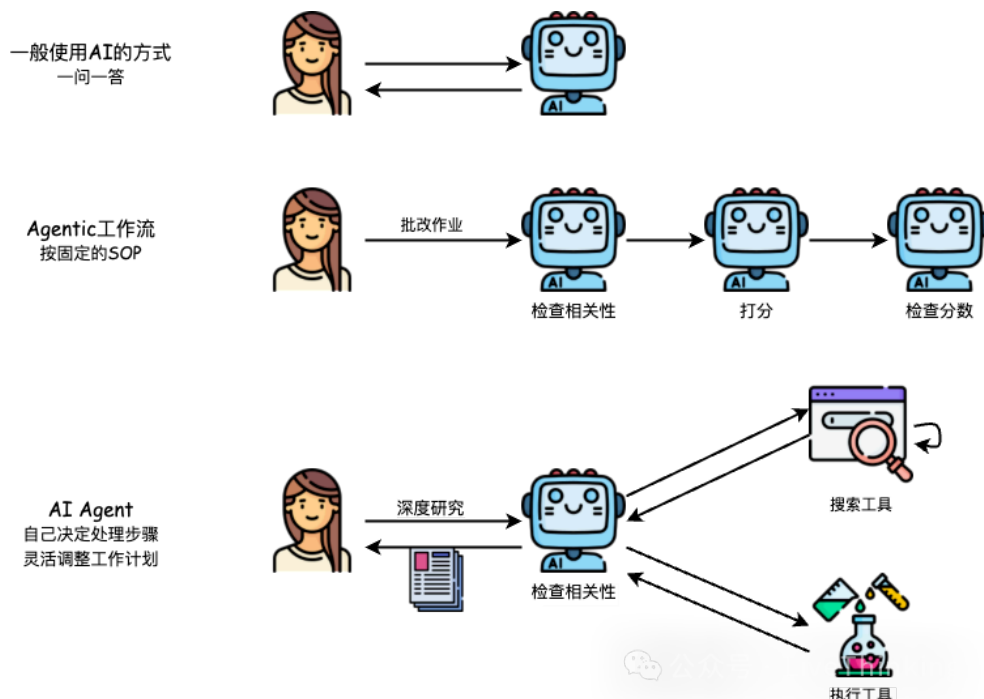
这些现实意味着，不仅是长度，深思熟虑的上下文对于构建强大的智能体至关重要。

更多的上下文失效的分析可以看这篇：[Context Engineering：长上下文是如何失效的？](#)

如何进行上下文工程(Context Engineering)？

Agentic模式的即时上下文策略

目前对模型的使用方式大致有以下几种：

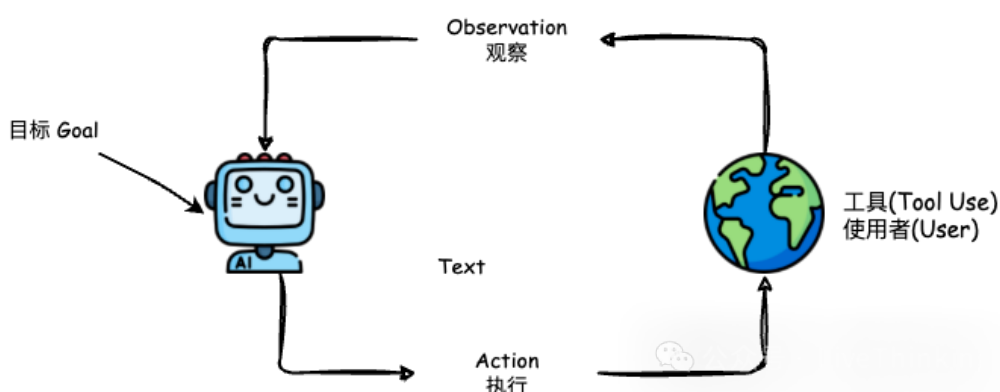


回顾一下：模型本身是无状态的，所有的上下文都需要由我们提供，或由工具在执行过程上提供。

而现在主流的智能体化的处理模式，其实就是一个循环，如Claude所讲：

在《构建有效的 AI 智能体》中，我们强调了基于 LLM 的工作流程与智能体之间的差异。自我们撰写那篇文章以来，我们逐渐倾向于一个对智能体的简单定义：**在一个循环中自主使用工具的 LLM**。

在这个过程中，大模型自主制定计划并且调用相关的工具完成指定的任务：



智能体的迭代处理流程

对于开发者，这个过程其实是可控的，而上下文工程就是在这个过程中，不断调整上下文，使上下文更加有效。

今天，许多 AI 原生应用都采用了某种**基于嵌入的预推理时间检索**来为智能体提供重要的上下文以供其推理。随着该领域向更具智能体化的方法过渡，我们越来越多地看到团队用“即时”上下文策略来增强这些检索系统。

与预先处理所有相关数据不同，采用“即时”方法构建的智能体维护轻量级标识符（文件路径、存储的查询、网页链接等），并使用这些引用在运行时使用工具动态地将数据加载到上下文中。

上下文处理的方法

为了生成有效(Effective)的上下文，Claude的博文在博文中提供了几种方案：

压缩(Compaction)

压缩是指将一个接近上下文窗口限制的对话，总结其内容，并用该摘要重新启动一个新的上下文窗口的做法。压缩通常是上下文工程中驱动更好长期连贯性的第一个杠杆。其核心是，压缩以高保真度的方式提炼上下文窗口的内容，使智能体能够以最小的性能下降继续工作。

压缩的核心就是减少上下文中不必要的信息，从而让模型得到更准确的上下文。

在这篇文章([Context Engineering: 如何修复上下文](#))中讲述了非常具体的方法：

- 工具装载(Tool Loadout): 仅选择相关的工具定义添加到上下文当中。
- 上下文修剪(Context Pruning): 从上下文中移除无关或不必要的信息。
- 上下文摘要(Context Summarization): 将累积的上下文浓缩为精简的摘要。
- 上下文卸载(Context Offloading): 将信息存储在大语言模型的上下文之外，通常通过用于存储和管理数据的工具来实现。

是进行压缩处理的各种方法。这种的权衡非常重要。

压缩的艺术在于选择保留什么与丢弃什么，因为过于激进的压缩可能会导致 subtle 但关键的上下文丢失，而其重要性只在稍后才显现出来。

在Manus的这篇论文([AI代理的上下文工程：构建Manus的经验教训](#))中，写的使用遮蔽(Mask)([深入学习Manus最新论文: 2个必学基础, 1个惊人洞察](#))，就是对工具装载的一种实现方式，因为工具的信息不仅是上下文的一部分(在调用接口中，工具是需要通过参数提供给模型的)。

而工具数量太多，也会给大模型造成困扰，从而在选择工具时无法精准选择。

这些都需要在迭代循环的过程中根据实际情况进行选择。

结构化笔记(Structured note-taking)

这点在Claude Code中有很强的感受。

在整个编程过程中，Claude Code会生成大量的markdown文件，比如技术要求、业务说明，还有过程中的Todo list，都是使用文件方式放在工作目录下。

这种策略以最小的开销提供了持久的记忆。就像 Claude Code 创建一个待办事项列表，或者您的自定义智能体维护一个 NOTES.md 文件一样，这个简单的模式允许智能体跟踪复杂任务的进度，保持关键的上下文和依赖关系，否则这些信息会在数十次工具调用中丢失。

个人认为，我们还有各种持久化的工具可以使用，如保存在SQL数据库、向量数据库或各种存储系统中。

而在保存，以及相应的检索时，都需要根据当前的具体情况，即时进行处理。

子智能体架构

在这篇文章中也提到相似的建议，即**上下文隔离(Context Quarantine)**：将不同的上下文分隔在各自专用的线程中。

它的核心是，将一个大的任务分解为小的任务，小的任务由一个专门的智能体完成，这样对于这个智能体来讲，只需要知道它需要知道的一些信息即可，即完成特定的任务。

在完成特定的任务后，将结果提交给一个主控的智能体对各个子智能体的数据进行汇总，分析，整合，最后生成最终的结果。

子智能体架构提供了另一种绕过上下文限制的方法。与其让一个智能体试图在整个项目中维护状态，不如让专门的子智能体用干净的上下文窗口处理集中的任务。主智能体用一个高层次的计划进行协调，而子智能体则执行深入的技术工作或使用工具来寻找相关信息。每个子智能体可能会进行广泛的探索，使用数万甚至更多的 token，但只返回一个浓缩、提炼过的工作总结（通常是 1,000-2,000 个 token）。

结论

不论是上下文工程(Context Engineering)，还是提示工程(Prompt Engineering)，目的都是向大模型提供更有针对性，更有价值的信息，在这种情况下大模型才能有更好的输出。

我们平时在使用各种聊天工具时，多了解一些提示工程，可以在不设置系统提示的情况得到更好的回答。

而在进行AI系统开发，尤其是智能体架构的开发时，针对业务特性进行的数据内容、数据保存形式等的设计至关重要，因为它是组织模型上下文的核心。

与本文相关的一些文章：

- [Context Engineering：不要构建多代理](#)
- [Context Engineering: 来自LangChain的剖析](#)
- [Context Engineering: 为提示词引入工程严谨性](#)
- [Context Engineering：长上下文是如何失效的？](#)
- [Context Engineering: 如何修复上下文](#)
- [AI代理的上下文工程：构建Manus的经验教训](#)
- [深入学习Manus最新论文: 2个必学基础，1个惊人洞察](#)

- [Anthropic: 如何构建多智能体研究系统](#)
- [Google: Agents白皮书](#)
- [OpenAI:构建Agent的实用指南](#)
- [Anthropic:构建有效的Agent](#)

引用链接

[1] Effective context engineering for AI

agents: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

[2] Chroma关于模型上下文内容研究: <https://research.trychroma.com/context-rot>

✌️ 感谢你花时间看到这里，欢迎留言跟我分享你的问题、想法和成果，并请关注一下。

👍 如果这篇文章对你有帮助，请点个"赞"和"在看"，这是对我的最大鼓励。

✈️ 也欢迎转发给朋友，与更多人一起在学习一起实践。

👉 在AI实践中如果遇到问题，欢迎来加我的微信[jackz02]进行交流，备注[vibe coding]。



J杰克